

Credit Card Approval Prediction System

Project Description:

Credit Card Approval Prediction System is a Machine Learning-based web application developed to assist banks and financial institutions in making faster and more consistent credit card approval decisions. The application analyzes applicant information such as gender, age, income, employment experience, education, family status, housing type, occupation, family members, and other financial details to predict whether a credit card application is likely to be approved or rejected.

The system was developed using Python, Flask, Pandas, NumPy, Scikit-learn, XGBoost, HTML, CSS, and Joblib. Multiple machine learning algorithms, including Logistic Regression, Decision Tree, Random Forest, and XGBoost, were trained and evaluated. Based on performance comparison, XGBoost was selected as the final prediction model because it achieved the best overall performance on the dataset.

The application provides a simple and user-friendly interface where users can enter applicant details and receive an instant prediction indicating whether the credit card application is approved or rejected. The trained model is integrated with a Flask web application and deployed on Render, allowing users to access the system through a web browser without any local installation.

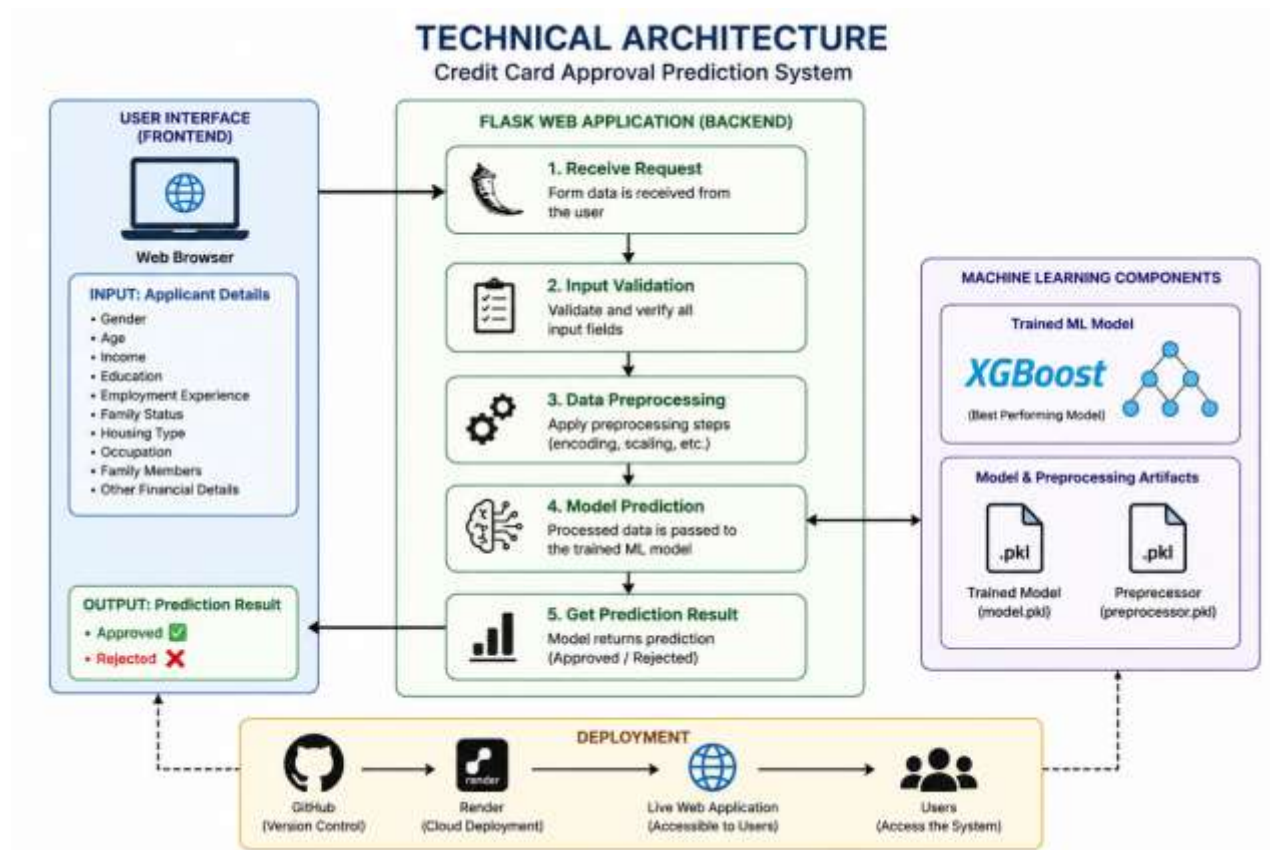
Scenarios

Scenario 1: A salaried employee with stable income, good employment history, and suitable financial details applies for a credit card. The system predicts that the application is likely to be approved, helping the bank process the application more quickly.

Scenario 2: A self-employed applicant with low income and poor credit history applies for a loan. The system predicts that the application is likely to be rejected, helping the bank identify high-risk applications early.

Scenario 3: A bank employee uses the Credit Card Approval Prediction System to evaluate multiple applications throughout the day. The application provides instant predictions through a simple web interface, reducing manual effort and improving the consistency of the approval process.

Technical Architecture:



Description:

The **Credit Card Approval Prediction System** follows a modular three-layer architecture consisting of a **Flask-based web application**, a **Machine Learning prediction engine**, and a **user interface**. The user enters applicant information through the web interface, which is sent to the Flask backend for processing. The backend validates the input data, applies the required preprocessing steps, and passes the processed data to the trained **XGBoost** model for prediction. The model returns the prediction result, which is then displayed to the user through the web interface.

The application uses Joblib to load the trained machine learning model and preprocessing files, ensuring fast and efficient predictions. The system is designed to provide a simple, reliable, and user-friendly solution for predicting whether a credit card application is likely to be approved or rejected.

Pre-requisites:

- Python Programming: [Python Documentation](#)
- Flask Framework: [Flask Documentation](#)
- Machine Learning Basics: [Scikit-learn Documentation](#)
- Pandas & NumPy: [Official Documentation](#)
- XGBoost Library: [XGBoost Documentation](#)
- Version Control with Git: [Git Documentation](#)
- Development Environment: [Visual Studio Code](#)
- Render Deployment: [Render Documentation](#)

Project Work low:

The Credit Card Approval Prediction System was developed by following a structured workflow that includes dataset preparation, machine learning model development, web application integration, and deployment. Each milestone represents a major phase in building the complete application.

Milestone 1: Model Selection and Architecture

- Activity 1.1: Collect and understand the Credit Card Approval dataset.
- Activity 1.2: Research, train, and compare different Machine Learning algorithms for credit card approval prediction.
- Activity 1.3: Design the overall system architecture, including the Flask application and Machine Learning model.
- Activity 1.4: Set up the development environment by installing the required software, libraries, and project dependencies.

Milestone 2: Core Functionalities Development

- Activity 2.1: Develop the credit card approval prediction functionality using the trained Machine Learning model.
- Activity 2.2: Integrate the Flask backend to handle user input, perform prediction, and display the prediction result.

Milestone 3: App.py Development

- Activity 3.1: Develop the main application logic in app.py, including model loading, input validation, prediction work low, and API integration.

Milestone 4: Frontend Development

- Activity 4.1: Design and develop the user interface using React, HTML, CSS, and JavaScript with a responsive layout.
- Activity 4.2: Create dynamic pages to display prediction results, confidence score, history, and downloadable PDF reports.

Milestone 5: Deployment

- Activity 5.1: Prepare the application for deployment by configuring the project, installing dependencies, and testing locally.
- Activity 5.2: Deploy the Smart Lender application on Render and verify that all features work correctly in the production environment.

Milestone 6: Conclusion

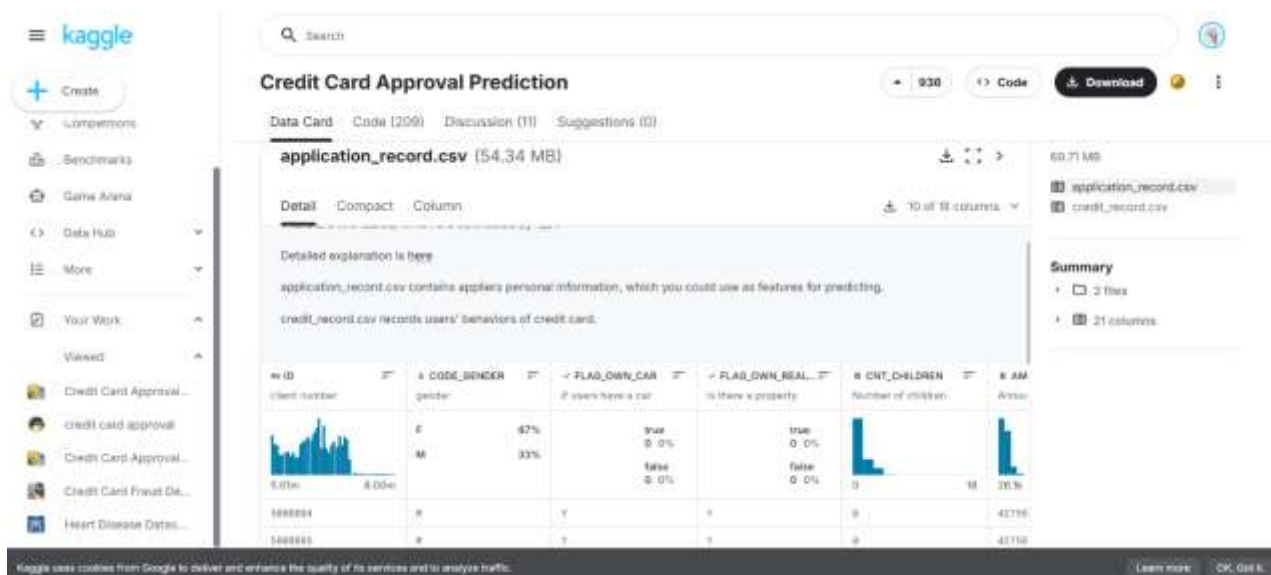
Milestone 1: Model Selection and Architecture

In this milestone, we focus on selecting the most suitable Machine Learning model for predicting loan approval. We also prepare the dataset, understand its features, design the system architecture, and configure the development environment required for building the Smart Lender application.

Activity 1.1: Collect and Understand the Loan Prediction Dataset

Before developing the Credit Card Approval Prediction System, a suitable dataset was collected to train and evaluate the Machine Learning models. The dataset contains applicant information such as gender, age, annual income, employment experience, family status, education level, housing type, occupation, number of family members, and other financial details. These features are used by the Machine Learning model to determine whether a credit card application is likely to be approved or rejected.

- **Step 1 – Download the Dataset:** Download the Credit Card Approval dataset from the provided source and store it in the project directory for further preprocessing and model development.



- **Step 2 – Explore the Dataset :** Open the dataset using Jupyter Notebook or Visual Studio Code to examine the available features, data types, missing values, and target variable. Understanding the dataset helps identify the preprocessing techniques required before training the Machine Learning models.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S
1	ID	CODE	GEI FLAG	OW FLAG	OW CNT	CHIL	AMT	INCC NAME	IN NAME	ED NAME	FA NAME	HC DAYS	BIR* DAYS	EMI FLAG	MGI FLAG	WO FLAG	PHC FLAG	EMV OCCUPATI	CNT_FAM_MEMBER
2	5008804	M	Y	Y	Y	0	427500	Working	Higher eds	Civil marri	Rented ap	-12005	-4542	1	1	0	0		2
3	5008805	M	Y	Y	Y	0	427500	Working	Higher eds	Civil marri	Rented ap	-12005	-4542	1	1	0	0		2
4	5008806	M	Y	Y	Y	0	112500	Working	Secondary	Married	House / at	-21474	-1134	1	0	0	0	Security at	2
5	5008808	F	N	Y	Y	0	270000	Commercial	Secondary	Single / no House	/ at	-19110	-3051	1	0	1	1	Sales staff	1
6	5008809	F	N	Y	Y	0	270000	Commercial	Secondary	Single / no House	/ at	-19110	-3051	1	0	1	1	Sales staff	1
7	5008810	F	N	Y	Y	0	270000	Commercial	Secondary	Single / no House	/ at	-19110	-3051	1	0	1	1	Sales staff	1
8	5008811	F	N	Y	Y	0	270000	Commercial	Secondary	Single / no House	/ at	-19110	-3051	1	0	1	1	Sales staff	1
9	5008812	F	N	Y	Y	0	283500	Pensioner	Higher eds	Separated	House / at	-22464	365243	1	0	0	0		1
10	5008813	F	N	Y	Y	0	283500	Pensioner	Higher eds	Separated	House / at	-22464	365243	1	0	0	0		1
11	5008814	F	N	Y	Y	0	283500	Pensioner	Higher eds	Separated	House / at	-22464	365243	1	0	0	0		1
12	5008815	M	Y	Y	Y	0	270000	Working	Higher eds	Married	House / at	-16872	-769	1	1	1	1	Accountan	2
13	5112956	M	Y	Y	Y	0	270000	Working	Higher eds	Married	House / at	-16872	-769	1	1	1	1	Accountan	2
14	6153651	M	Y	Y	Y	0	270000	Working	Higher eds	Married	House / at	-16872	-769	1	1	1	1	Accountan	2
15	5008819	M	Y	Y	Y	0	135000	Commercial	Secondary	Married	House / at	-17778	-1194	1	0	0	0	Laborers	2
16	5008820	M	Y	Y	Y	0	135000	Commercial	Secondary	Married	House / at	-17778	-1194	1	0	0	0	Laborers	2
17	5008821	M	Y	Y	Y	0	135000	Commercial	Secondary	Married	House / at	-17778	-1194	1	0	0	0	Laborers	2
18	5008822	M	Y	Y	Y	0	135000	Commercial	Secondary	Married	House / at	-17778	-1194	1	0	0	0	Laborers	2
19	5008823	M	Y	Y	Y	0	135000	Commercial	Secondary	Married	House / at	-17778	-1194	1	0	0	0	Laborers	2
20	5008824	M	Y	Y	Y	0	135000	Commercial	Secondary	Married	House / at	-17778	-1194	1	0	0	0	Laborers	2
21	5008825	F	Y	N	Y	0	130500	Working	Incomplete	Married	House / at	-10669	-1103	1	0	0	0	Accountan	2
22	5008826	F	Y	N	Y	0	130500	Working	Incomplete	Married	House / at	-10669	-1103	1	0	0	0	Accountan	2
23	5008830	F	N	Y	Y	0	157500	Working	Secondary	Married	House / at	-10031	-1469	1	0	1	0	Laborers	2
24	5008831	F	N	Y	Y	0	157500	Working	Secondary	Married	House / at	-10031	-1469	1	0	1	0	Laborers	2
25	5008832	F	N	Y	Y	0	157500	Working	Secondary	Married	House / at	-10031	-1469	1	0	1	0	Laborers	2
26	5008834	F	N	Y	Y	1	112500	Working	Secondary	Single / no House	/ at	-10968	-1620	1	0	0	0		2

- **Step 3 - Analyze Dataset Features:** Study the dataset features to understand how each applicant attribute contributes to the credit card approval prediction. The target variable indicates whether the credit card application is approved or rejected, providing the basis for training and evaluating different Machine Learning models.

```

Application Shape : (438557, 18)
Credit Shape : (1048575, 3)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 438557 entries, 0 to 438556
Data columns (total 18 columns):
#   Column                Non-Null Count  Dtype
---  -
0   ID                     438557 non-null  int64
1   CODE_GENDER           438557 non-null  object
2   FLAG_OWN_CAR          438557 non-null  object
3   FLAG_OWN_REALTY       438557 non-null  object
4   CNT_CHILDREN          438557 non-null  int64
5   AMT_INCOME_TOTAL     438557 non-null  float64
6   NAME_INCOME_TYPE      438557 non-null  object
7   NAME_EDUCATION_TYPE   438557 non-null  object
8   NAME_FAMILY_STATUS    438557 non-null  object
9   NAME_HOUSING_TYPE     438557 non-null  object
10  DAYS_BIRTH            438557 non-null  int64
11  DAYS_EMPLOYED         438557 non-null  int64
12  FLAG_MOBIL            438557 non-null  int64
13  FLAG_WORK_PHONE       438557 non-null  int64
14  FLAG_PHONE            438557 non-null  int64
15  FLAG_EMAIL            438557 non-null  int64
16  OCCUPATION_TYPE       304354 non-null  object
17  CNT_FAM_MEMBERS       438557 non-null  float64
...
1   MONTHS_BALANCE       1048575 non-null  int64
2   STATUS               1048575 non-null  object
dtypes: int64(2), object(1)
memory usage: 24.0+ MB

```

Activity 1.2: Research and Select the Appropriate Machine Learning Model

Several Machine Learning algorithms were studied and evaluated before selecting the final model for the Credit Card Approval Prediction System. Each algorithm was trained using the prepared dataset and compared based on its prediction performance. The objective was to identify the model that provides accurate and reliable predictions for credit card approval.

The following algorithms were evaluated:

- Logistic Regression
- Decision Tree
- Random Forest
- XGBoost

Each model was trained and tested using the same dataset to ensure a fair comparison. The evaluation was performed using standard classification metrics such as Accuracy, Precision, Recall, F1-Score, and ROC-AUC Score.

After comparing the performance of all models, XGBoost was selected as the final prediction model because it achieved the best overall performance and provided more reliable prediction results than the other algorithms.

Activity 1.3: Define the Architecture of the Application

The Credit Card Approval Prediction System follows a modular architecture consisting of three main layers:

- Frontend Layer: Developed using HTML and CSS to collect applicant information and display prediction results.
- Backend Layer: Developed using Flask to validate user input, load the trained Machine Learning model, perform preprocessing, and generate predictions.
- Machine Learning Layer: Uses the trained XGBoost model to predict loan approval probability based on applicant details.
- Database Layer: SQLite is used to store prediction history.
- Reporting Layer: ReportLab generates downloadable PDF reports for each prediction.

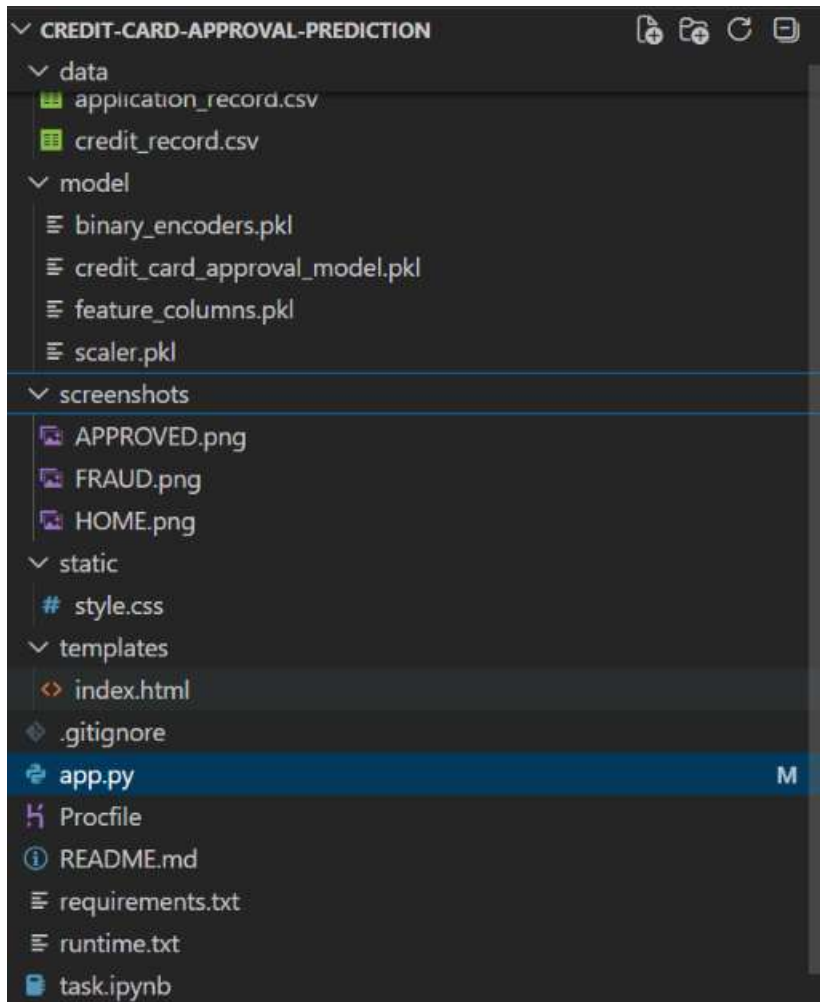
This architecture keeps the application modular, maintainable, and easy to deploy.

Activity 1.4: Set Up the Development Environment :

Before development, the required software and libraries were installed.

Development Environment:

- Python 3.x
- Visual Studio Code
- Jupyter Notebook
- Git & GitHub
- Flask
- Pandas
- NumPy
- Scikit-learn
- XGBoost
- Joblib
- Render



Milestone 2: Core Functionalities Development:

In this milestone, the core functionalities of the Credit Card Approval Prediction System were developed. The application was designed to collect applicant information, validate user input, process the data using the trained Machine Learning model, and display the prediction result through a simple and user-friendly web interface. The Flask backend was integrated with the frontend to ensure smooth communication between the user interface and the prediction model.

Activity 2.1: Develop Core Content Generation Features

The main features of the Smart Lender application include:

Loan Approval Prediction

The application collects applicant information such as income, education, employment status, loan amount, loan term, credit history, and property area. After validating the input, the trained XGBoost model predicts whether the loan application is likely to be approved or rejected.

Approval Probability

Along with the prediction result, the application displays the confidence score (approval probability), helping users understand the model's confidence in its prediction.

Prediction History

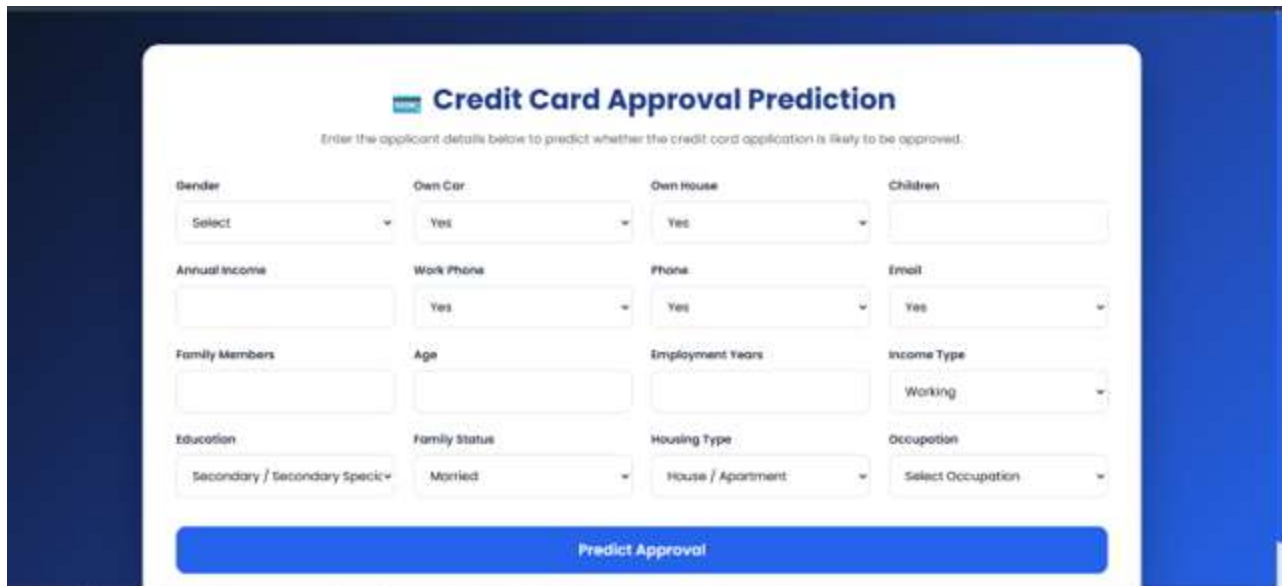
Every prediction is stored in the SQLite database. Users can view previous loan evaluations, making it easy to track and review earlier predictions.

PDF Report Generation

The application generates a professional PDF report containing the applicant details, prediction result, probability score, and evaluation summary. Users can download the report for future reference.

User-Friendly Interface

A clean and responsive interface allows users to easily enter applicant details, submit the form, view prediction results, and download reports.

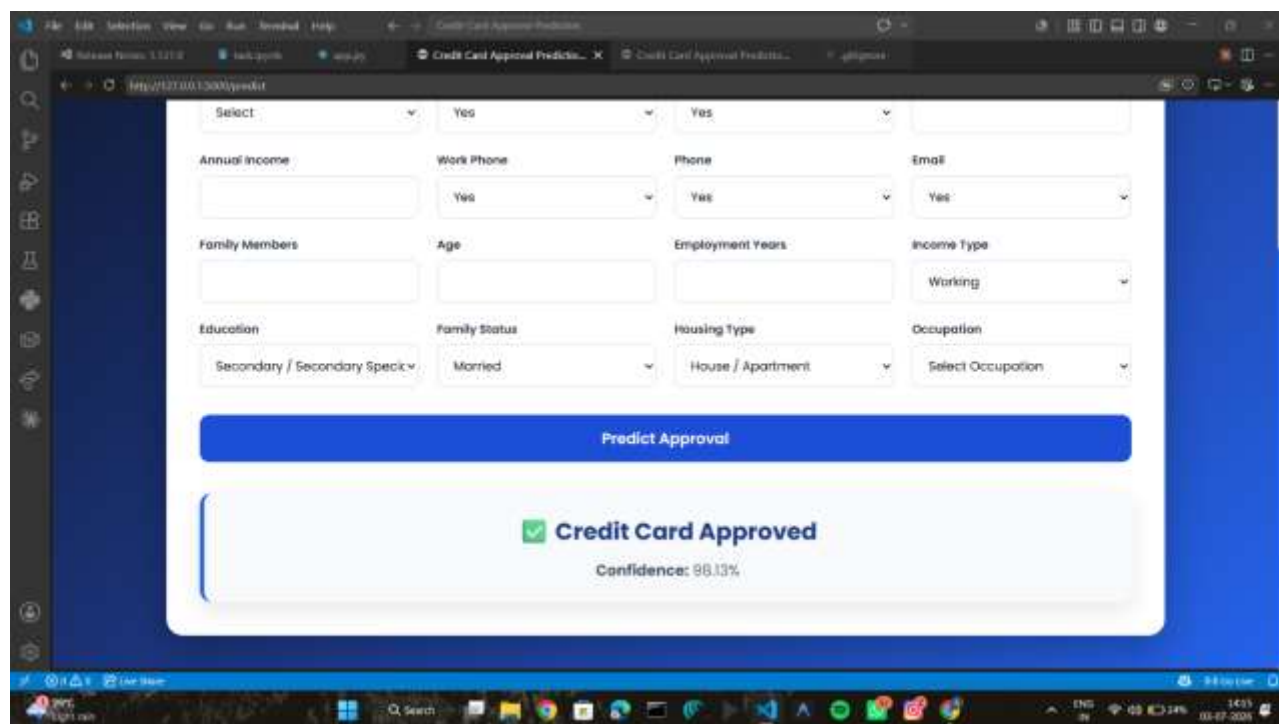


The screenshot shows a web form titled "Credit Card Approval Prediction" with a subtitle "Enter the applicant details below to predict whether the credit card application is likely to be approved." The form is organized into a grid of input fields:

Gender	Own Car	Own House	Children
Select	Yes	Yes	
Annual Income	Work Phone	Phone	Email
	Yes	Yes	Yes
Family Members	Age	Employment Years	Income Type
			Working
Education	Family Status	Housing Type	Occupation
Secondary / Secondary Spec	Married	House / Apartment	Select Occupation

At the bottom of the form is a large blue button labeled "Predict Approval".

Prediction Result Page :



The screenshot shows a web browser window displaying a "Credit Card Approval Prediction" application. The interface includes a sidebar with navigation icons and a main content area with a form and a results section.

Form Fields:

- Select:** A dropdown menu.
- Annual Income:** A text input field.
- Family Members:** A text input field.
- Education:** A dropdown menu with "Secondary / Secondary Spec" selected.
- Work Phone:** A dropdown menu with "Yes" selected.
- Age:** A text input field.
- Family Status:** A dropdown menu with "Married" selected.
- Phone:** A dropdown menu with "Yes" selected.
- Employment Years:** A text input field.
- Housing Type:** A dropdown menu with "House / Apartment" selected.
- Email:** A dropdown menu with "Yes" selected.
- Income Type:** A dropdown menu with "Working" selected.
- Occupation:** A dropdown menu with "Select Occupation" selected.

Action Button: A large blue button labeled "Predict Approval".

Results Section:

- A green checkmark icon.
- Credit Card Approved**
- Confidence: 86.13%

The browser's address bar shows the URL: <http://127.0.0.1:3000/index.html>. The Windows taskbar at the bottom shows the system clock as 14:35 on 03-07-2021.

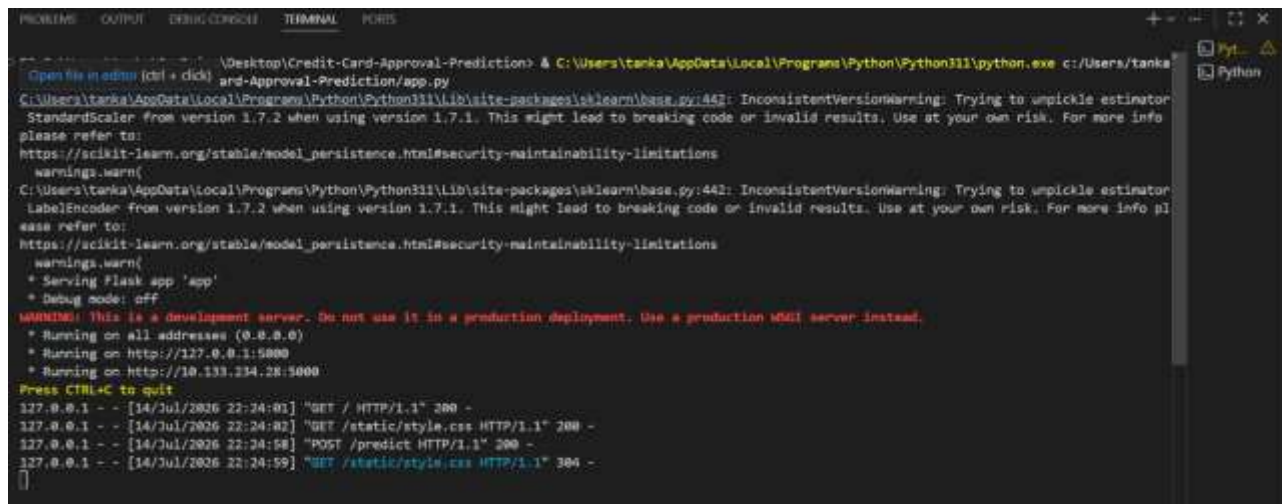
Activity 2.2: Implement the Flask Backend

The Flask backend manages all server-side operations of the Credit Card Approval Prediction System.

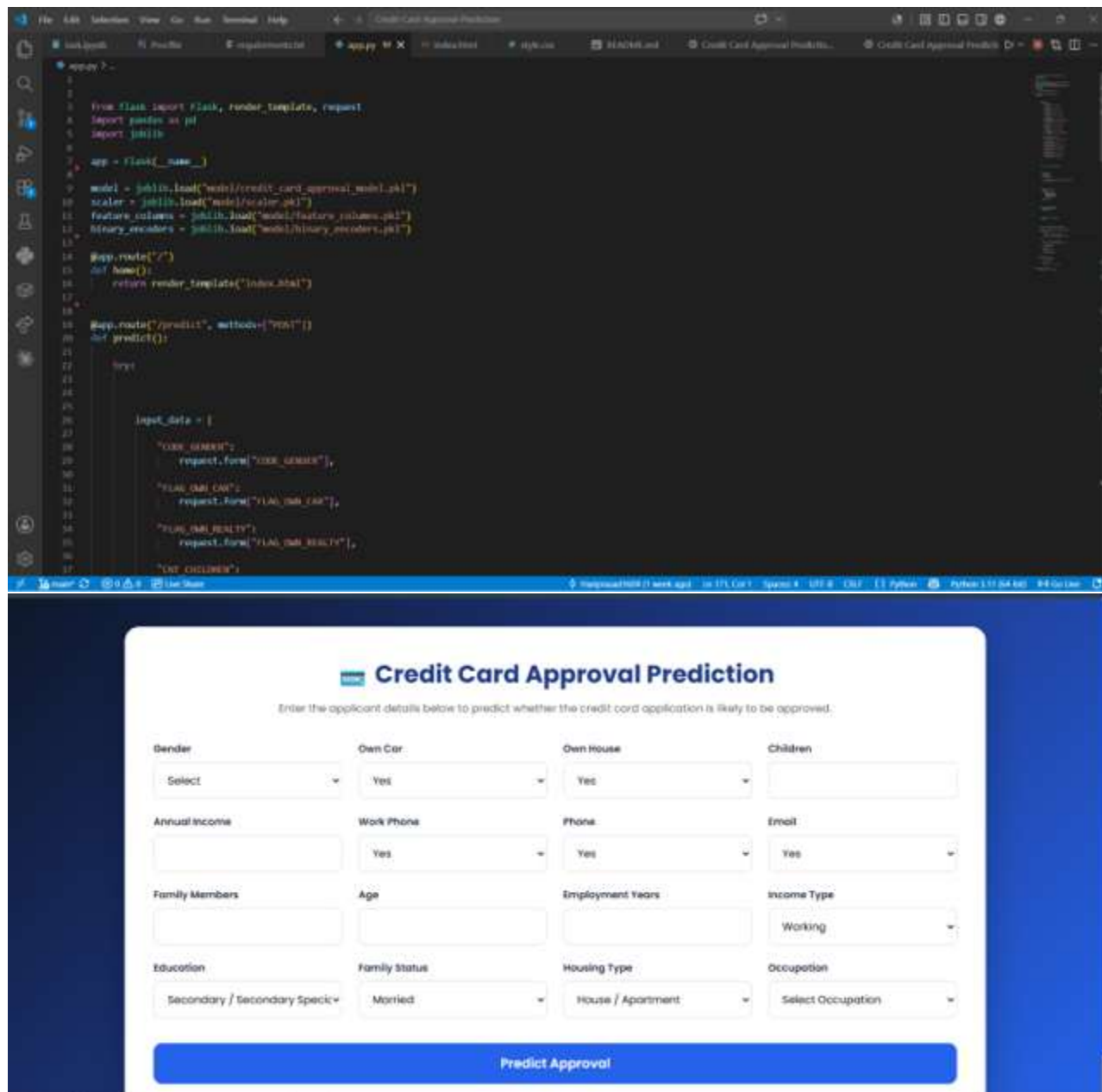
The backend performs the following tasks:

- Receives applicant information submitted through the web form.
- Validates all required input fields before prediction.
- Loads the trained Machine Learning model and preprocessing files.
- Applies the necessary preprocessing to the input data.
- Performs credit card approval prediction using the trained XGBoost model.
- Returns the prediction result to the frontend for display.

This backend architecture ensures secure data processing, fast prediction, and smooth communication between the frontend and the Machine Learning model.



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Open File in Editor (ctrl + click)
C:\Users\tanka\AppData\Local\Programs\Python\Python311\python.exe c:/Users/tanka/
ard-Approval-Prediction/app.py
C:\Users\tanka\AppData\Local\Programs\Python\Python311\Lib\site-packages\sklearn\base.py:442: InconsistentVersionWarning: Trying to unpickle estimator
StandardScaler from version 1.7.2 when using version 1.7.1. This might lead to breaking code or invalid results. Use at your own risk. For more info pl
please refer to:
https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
warnings.warn(
C:\Users\tanka\AppData\Local\Programs\Python\Python311\Lib\site-packages\sklearn\base.py:442: InconsistentVersionWarning: Trying to unpickle estimator
LabelEncoder from version 1.7.2 when using version 1.7.1. This might lead to breaking code or invalid results. Use at your own risk. For more info pl
ease refer to:
https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
warnings.warn(
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://10.133.334.28:5000
Press CTRL+C to quit
127.0.0.1 - - [14/Jul/2026 22:24:01] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [14/Jul/2026 22:24:02] "GET /static/style.css HTTP/1.1" 200 -
127.0.0.1 - - [14/Jul/2026 22:24:58] "POST /predict HTTP/1.1" 200 -
127.0.0.1 - - [14/Jul/2026 22:24:59] "GET /static/style.css HTTP/1.1" 304 -
```



Milestone 3: App.py Development

This milestone focuses on developing the main application logic using **app.py**, which acts as the central controller of the Credit Card Approval Prediction System. It handles user requests, validates the input data, loads the trained Machine Learning model, performs credit card approval prediction, and returns the prediction result to the web interface.

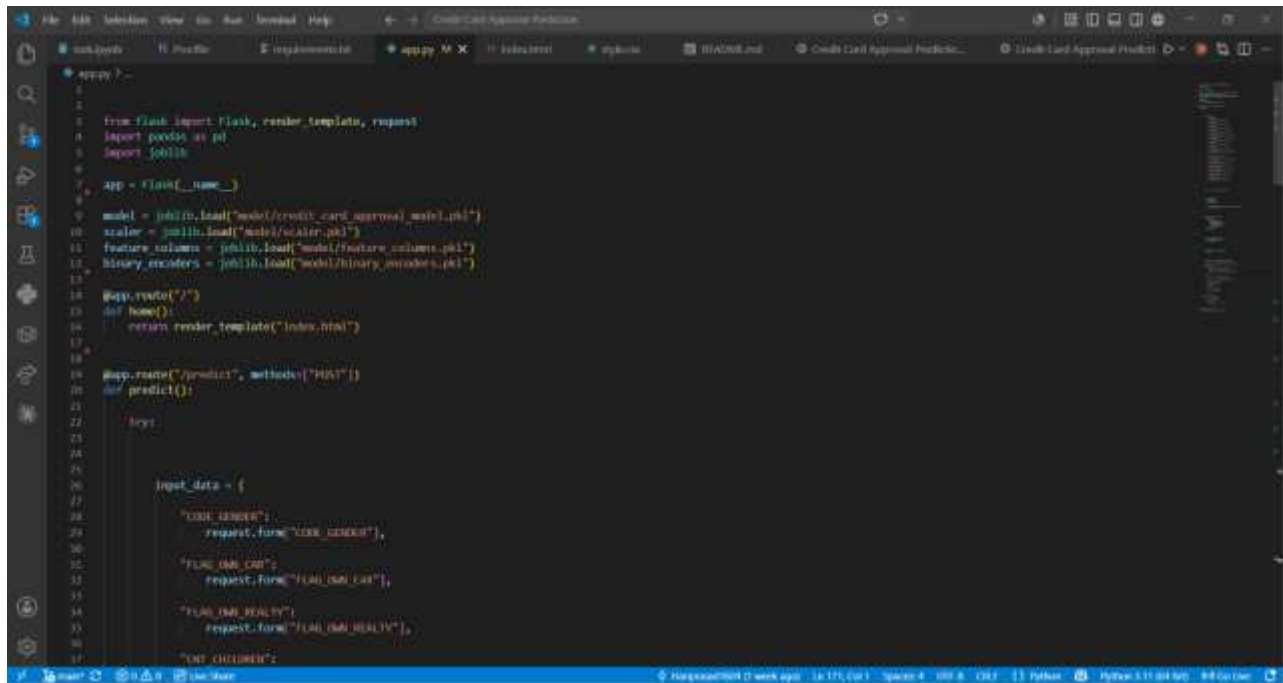
Activity 3.1: Develop the Main Application Logic

The **app.py** file connects all major components of the application, including the user interface, Machine Learning model, preprocessing files, and prediction workflow.

The main responsibilities of the application include:

- Receiving applicant information from the web form.
- Validating user input before prediction.

- Loading the trained Machine Learning model and preprocessing files.
- Converting user input into the required model format.
- Applying preprocessing to the input data.
- Predicting the credit card approval status.
- Returning the prediction result to the frontend.



```

1  # app.py
2
3  from flask import Flask, render_template, request
4  import pickle as pl
5  import joblib
6
7  app = Flask(__name__)
8
9  model = joblib.load("model/credit_card_approval_model.pkl")
10 scaler = joblib.load("model/scaler.pkl")
11 feature_columns = joblib.load("model/feature_columns.pkl")
12 binary_encoders = joblib.load("model/binary_encoders.pkl")
13
14 @app.route("/")
15 def home():
16     return render_template("index.html")
17
18 @app.route("/predict", methods=["POST"])
19 def predict():
20
21     # Keys
22
23     input_data = {
24
25         "GENDER":
26             request.form["GENDER"],
27
28         "FLAG_0M_CAR":
29             request.form["FLAG_0M_CAR"],
30
31         "FLAG_0M_REALTY":
32             request.form["FLAG_0M_REALTY"],
33
34         "ONT_CHILDREN":
35

```

Full app.py: <https://github.com/Hariprasad1604/Credit-Card-Approval-Prediction/blob/main/app.py>

Define the Core Routes in app.py

The Flask application provides different routes to manage the complete prediction workflow.

The important routes include:

- Home Route – Displays the home page of the Smart Lender application.
- Prediction Route – Accepts applicant details and performs loan approval prediction.
- History Route – Displays all previous prediction records stored in the database.
- PDF Report Route – Generates and downloads a PDF report for the selected prediction.

Each route performs a specific task, making the application modular and easy to maintain.

```
4 @app.route("/")
5 def home():
6     return render_template("index.html")
7
8
9 @app.route("/predict", methods=["POST"])
10 def predict():
11
12     try:
13
14         input_data = {
15
16             "CODE_GENDER":
17                 request.form["CODE_GENDER"],
18
19             "FLAG_OWN_CAR":
20                 request.form["FLAG_OWN_CAR"],
21
22             "FLAG_OWN_REALTY":
23                 request.form["FLAG_OWN_REALTY"],
24
25             "CNT_CHILDREN":
26                 int(request.form["CNT_CHILDREN"]),
27
28             "AMT_INCOME_TOTAL":
29                 float(request.form["AMT_INCOME_TOTAL"]),
```

Model Loading and Prediction Process

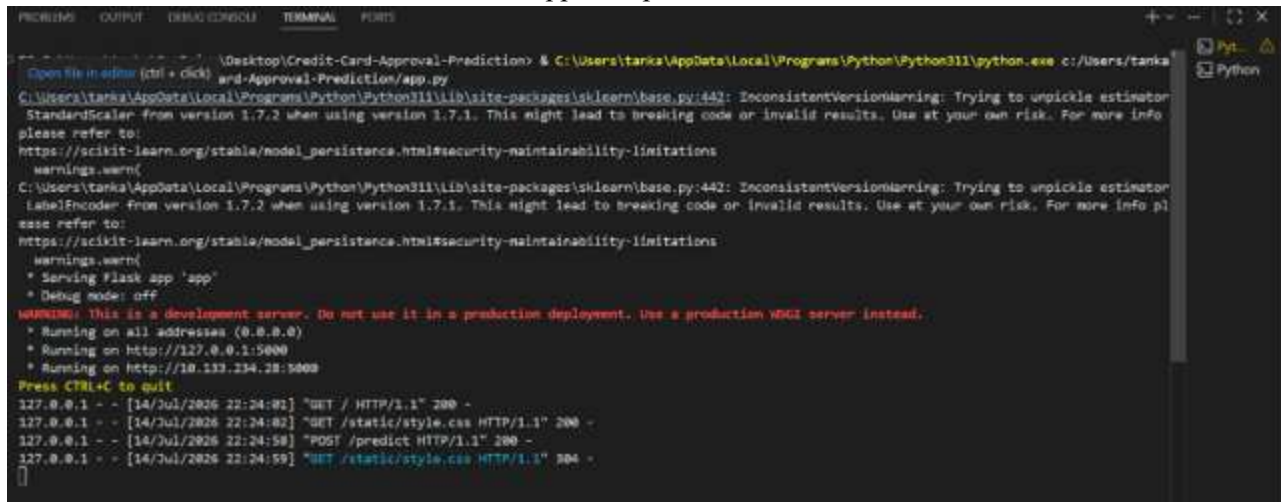
When the application starts, the trained **XGBoost** model and the required preprocessing files are loaded into memory using **Joblib**. This allows the application to generate predictions quickly without repeatedly loading the model for every request.

The prediction process follows these steps:

1. Receive applicant details from the web interface.
2. Validate the input values.
3. Convert the input into the required format.
4. Apply preprocessing using the saved preprocessing objects.
5. Pass the processed data to the trained XGBoost model.

6. Generate the prediction result.
7. Display the prediction result on the web page.

This work low ensures fast and reliable loan approval predictions.



```

C:\Users\tanka\AppData\Local\Programs\Python\Python311\python.exe c:/Users/tanka/.../Credit-Card-Approval-Prediction/app.py
C:\Users\tanka\AppData\Local\Programs\Python\Python311\lib\site-packages\sklearn\base.py:442: InconsistentVersionWarning: Trying to unpickle estimator
StandardScaler from version 1.7.2 when using version 1.7.1. This might lead to breaking code or invalid results. Use at your own risk. For more info pl
please refer to:
https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
warnings.warn(
C:\Users\tanka\AppData\Local\Programs\Python\Python311\lib\site-packages\sklearn\base.py:442: InconsistentVersionWarning: Trying to unpickle estimator
LabelEncoder from version 1.7.2 when using version 1.7.1. This might lead to breaking code or invalid results. Use at your own risk. For more info pl
please refer to:
https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
warnings.warn(
* Serving flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://10.133.234.28:5000
Press CTRL+C to quit
127.0.0.1 - - [14/Jul/2026 22:24:02] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [14/Jul/2026 22:24:02] "GET /static/style.css HTTP/1.1" 200 -
127.0.0.1 - - [14/Jul/2026 22:24:58] "POST /predict HTTP/1.1" 200 -
127.0.0.1 - - [14/Jul/2026 22:24:58] "GET /static/style.css HTTP/1.1" 204 -

```

Input Validation

Before making a prediction, the application verifies that all required fields are entered correctly.

The validation process includes:

- Checking for missing values.
- Ensuring numerical fields contain valid numbers.
- Validating loan amount and income values.
- Preventing invalid or empty form submissions.

This helps improve the reliability of the prediction system.

Prediction Response

After processing the applicant details, the system returns:

- Loan Approval Status (Approved / Rejected)
- Approval Probability
- Prediction Summary
- Option to Download PDF Report
- Option to View Prediction History

The response is displayed immediately on the results page, providing users with a clear understanding of the prediction outcome.

Milestone 4: Frontend Development

This milestone focuses on designing and developing the user interface of the Smart Lender application. The frontend was built to provide a clean, responsive, and user-friendly experience, allowing users to enter loan applicant details, view prediction results, access prediction history, and download PDF reports.

Activity 4.1: Design and Develop the User Interface

The user interface was designed using React, HTML, CSS, and JavaScript with a responsive layout that works smoothly across different screen sizes.

The main interface includes:

- Home Page: Introduces the Smart Lender application and its key features.
- Loan Prediction Form: Allows users to enter applicant details required for prediction.
- Prediction Results: Displays the loan approval status, approval probability, and prediction summary.
- Prediction History: Shows previously evaluated loan applications stored in the database.
- PDF Report Download: Allows users to download a detailed report for each prediction.

The interface was designed with a clean layout, simple navigation, and easy-to-understand labels to improve user experience.

Design a Responsive Layout Using CSS

The frontend was styled using modern CSS techniques to create a responsive and visually appealing interface.

The design includes:

- Responsive layout for desktop and mobile devices.
- Well-organized cards and forms and Proper spacing and alignment.
- Interactive buttons and navigation.
- Easy-to-read prediction results.

These design improvements make the application simple to use while maintaining a professional appearance.

Activity 4.2: Creating Dynamic Content Rendering with JavaScript

The frontend dynamically updates the interface based on the prediction results returned by the backend.

After the user submits the loan application:

- A loading indicator is displayed while the prediction is being processed.
- The loan approval result is shown instantly after prediction.
- The approval probability is displayed using visual components.
- Users can access prediction history without re-entering data.
- The PDF report download option becomes available after prediction.

The application communicates with the Flask backend through REST APIs, ensuring that the displayed information is always based on real prediction data.

Dynamic Features Implemented

The Smart Lender frontend includes several dynamic features that improve usability:

- Real-time display of prediction results.
- Dynamic rendering of approval probability.
- Navigation between different application pages.
- Automatic update of prediction history.
- Downloadable PDF reports.
- Responsive interface with smooth user interactions.

These features provide a better user experience and make the application more efficient and practical for real-world use.

Milestone 5: Deployment

This milestone focuses on deploying the Smart Lender application so that it can be accessed through a web browser from anywhere. Before deployment, the application was tested locally to ensure all features worked correctly. After successful testing, the project was deployed on Render, allowing users to access the application online.

Activity 5.1: Run the Application Locally

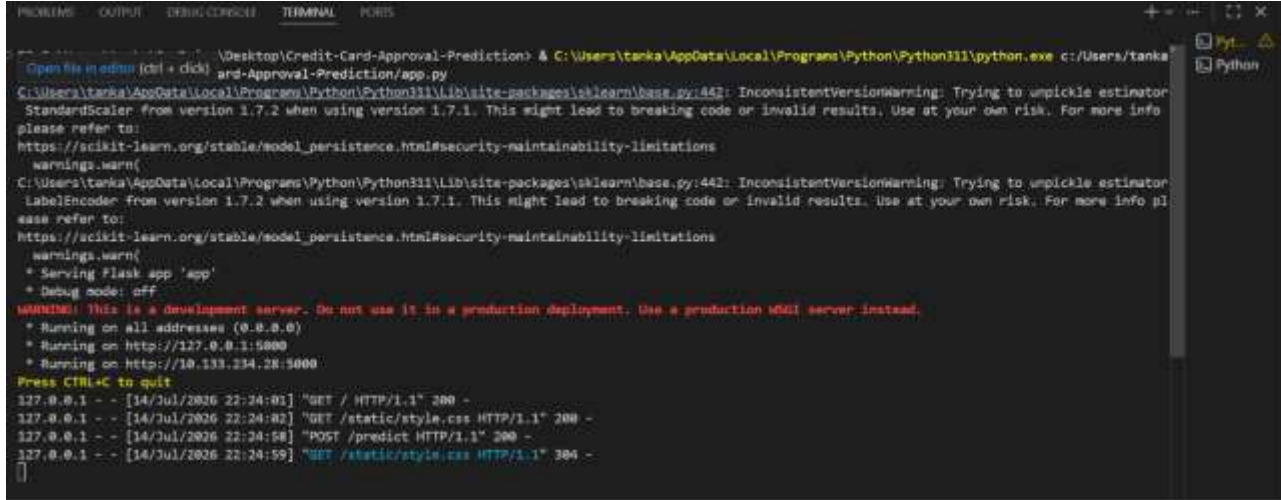
Before deploying the application, the project was tested on the local system to verify that all components were working correctly.

The following steps were performed:

- Created and activated the Python virtual environment.
- Installed all required project dependencies using the **requirements.txt** file.
- Started the Flask application locally.
- Verified that the prediction functionality worked correctly.

- Ensured that the web interface displayed the prediction results successfully.

Successful local testing helped identify and resolve issues before deployment.



```

C:\Users\tanka\Desktop\Credit-Card-Approval-Prediction> & C:\Users\tanka\AppData\Local\Programs\Python\Python311\python.exe c:/Users/tanka/
Open file in editor (ctrl + click) and-Approval-Prediction/app.py
C:\Users\tanka\AppData\Local\Programs\Python\Python311\Lib\site-packages\sklearn\base.py:442: InconsistentVersionWarning: Trying to unpickle estimator
StandardScaler from version 1.7.2 when using version 1.7.1. This might lead to breaking code or invalid results. Use at your own risk. For more info pl
please refer to:
https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
warnings.warn(
C:\Users\tanka\AppData\Local\Programs\Python\Python311\Lib\site-packages\sklearn\base.py:442: InconsistentVersionWarning: Trying to unpickle estimator
LabelEncoder from version 1.7.2 when using version 1.7.1. This might lead to breaking code or invalid results. Use at your own risk. For more info pl
ease refer to:
https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
warnings.warn(
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://10.133.234.28:5000
Press CTRL+C to quit
127.0.0.1 - - [14/Jul/2026 22:24:01] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [14/Jul/2026 22:24:02] "GET /static/style.css HTTP/1.1" 200 -
127.0.0.1 - - [14/Jul/2026 22:24:08] "POST /predict HTTP/1.1" 200 -
127.0.0.1 - - [14/Jul/2026 22:24:59] "GET /static/style.css HTTP/1.1" 304 -
  
```

Activity 5.2: Test the Application

After running the application locally, different applicant details were tested to verify that the prediction system worked correctly.

The testing process included:

- Verifying all input fields.
- Testing different applicant information.
- Checking both **Approved** and **Rejected** prediction results.
- Verifying that the prediction result was displayed correctly.
- Testing the application on different web browsers.

These tests ensured that the application functioned correctly before deployment.

Activity 5.3: Deploy the Application on Render

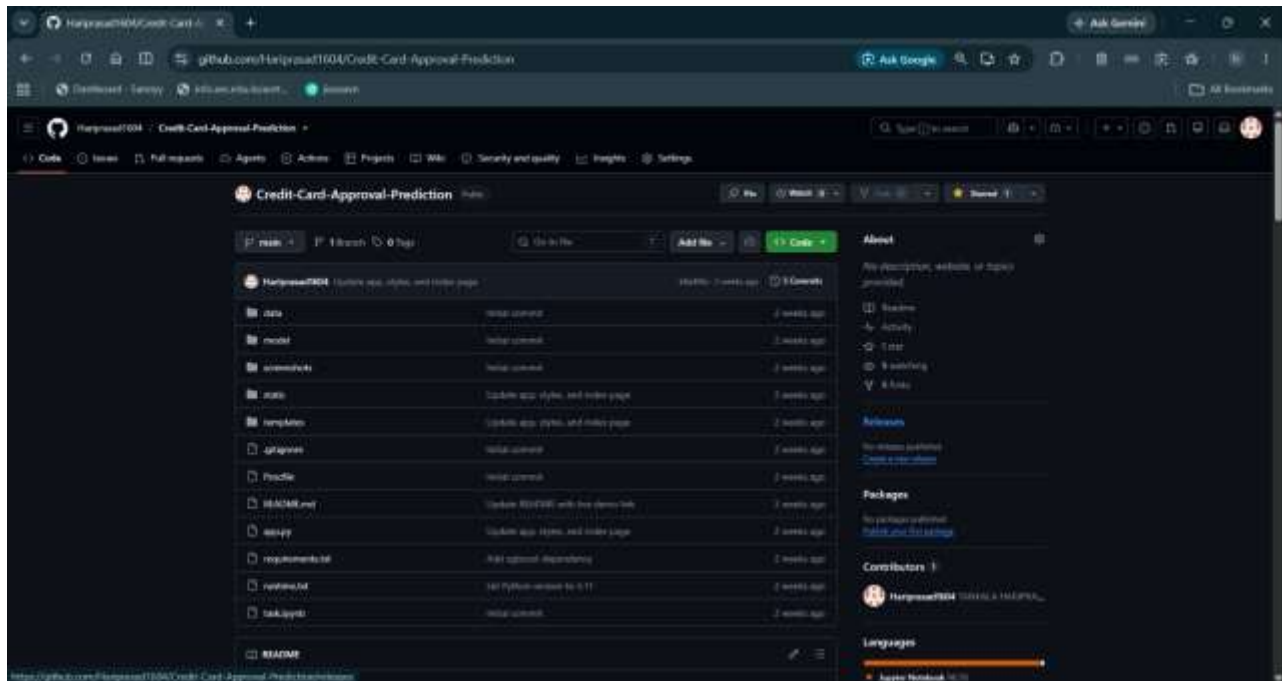
After successful local testing, the Credit Card Approval Prediction System was deployed on **Render**.

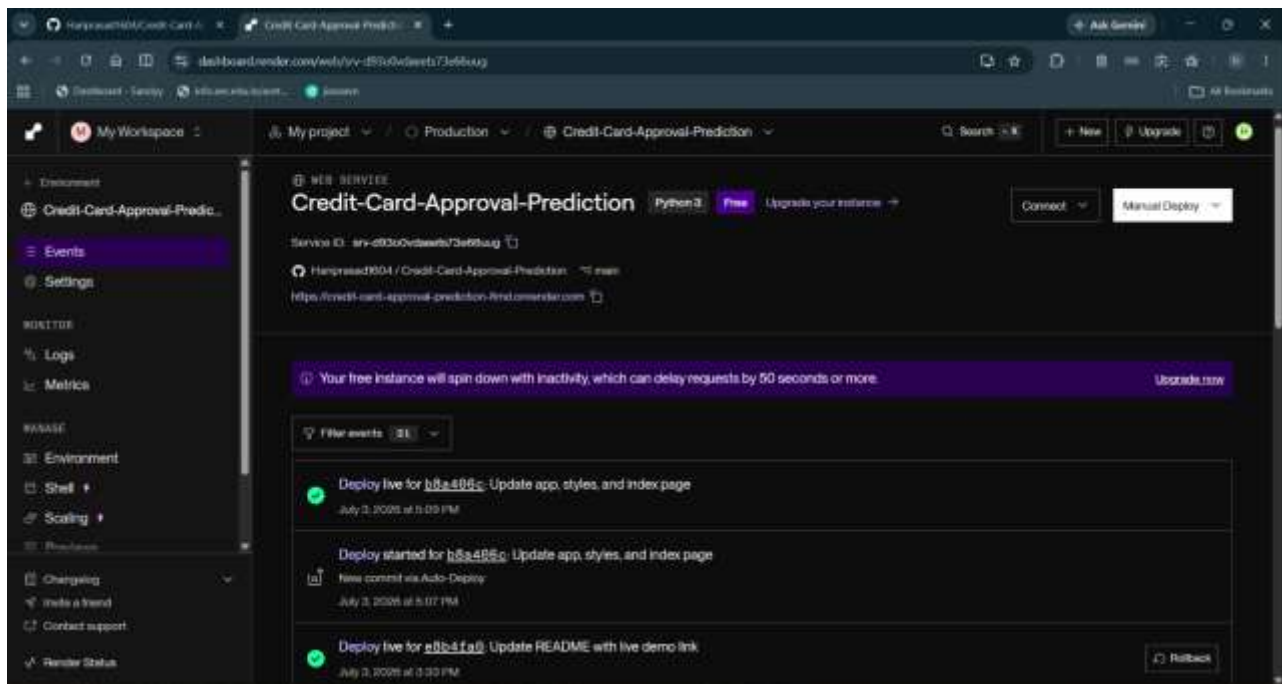
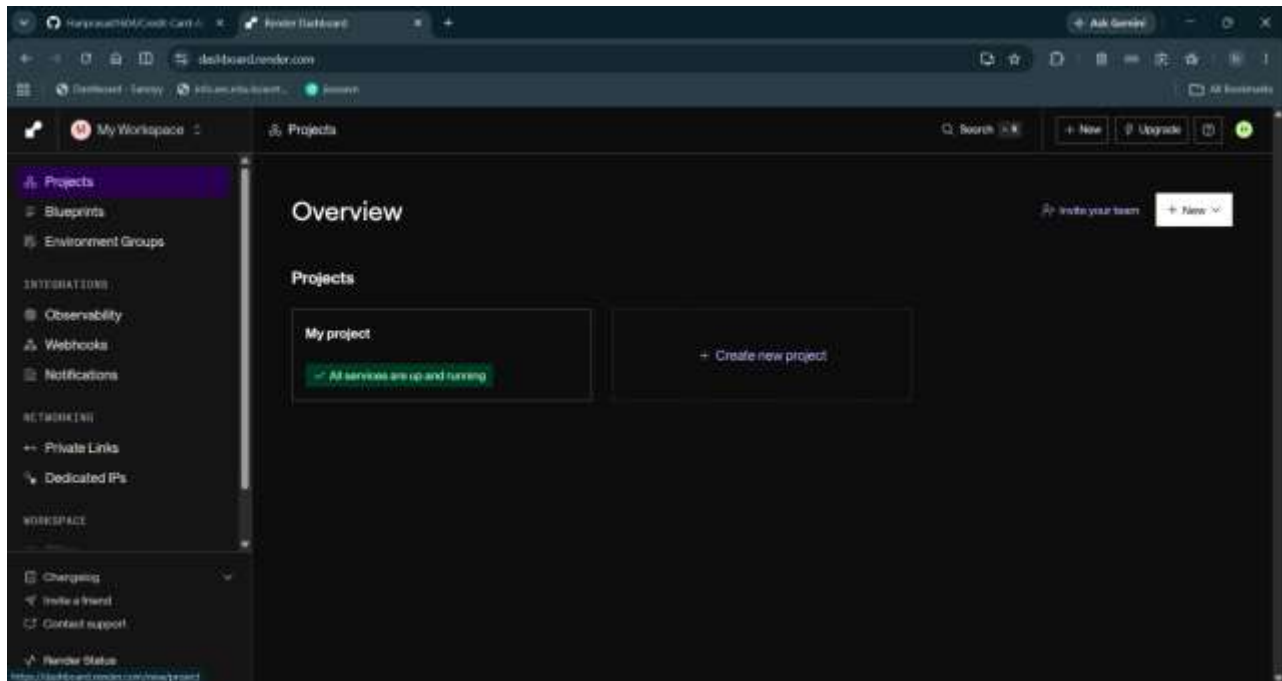
The deployment process included:

- Uploading the project to GitHub.
- Creating a new Web Service on Render.
- Connecting the GitHub repository with Render.
- Configuring the build and start commands.

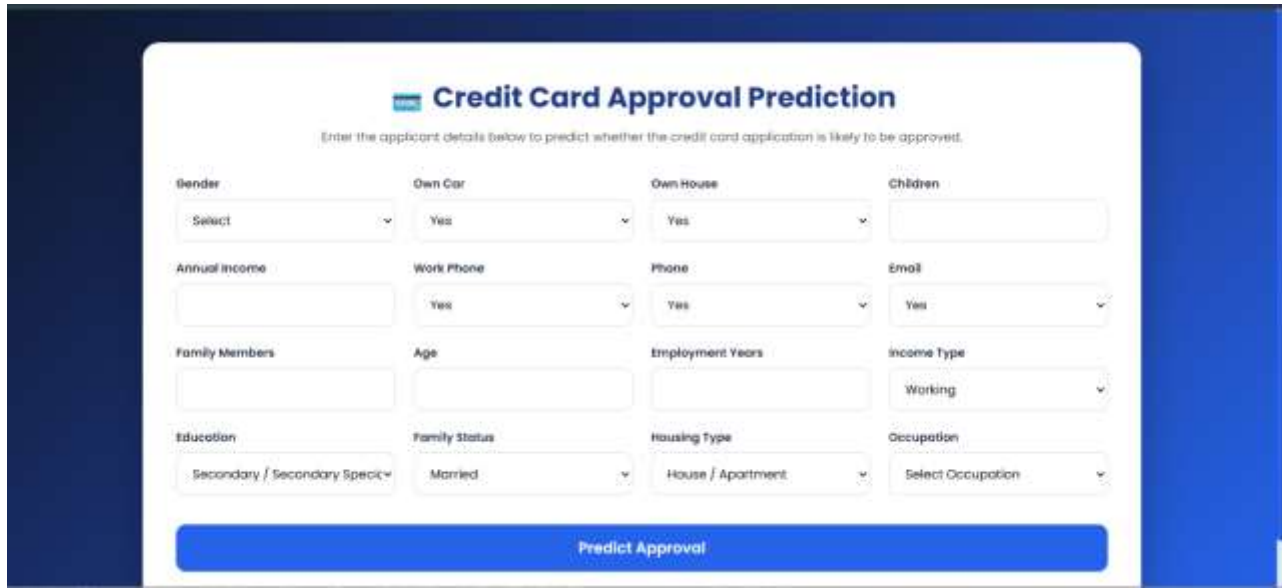
- Deploying the application.
- Verifying that the deployed application worked correctly.

The deployed application can now be accessed through a public URL, allowing users to test the system directly from their web browser.





deployment link: <https://credit-card-approval-prediction-ltmd.onrender.com>



Credit Card Approval Prediction

Enter the applicant details below to predict whether the credit card application is likely to be approved.

Gender Select	Own Car Yes	Own House Yes	Children
Annual Income 	Work Phone Yes	Phone Yes	Email Yes
Family Members 	Age 	Employment Years 	Income Type Working
Education Secondary / Secondary Spec	Family Status Married	Housing Type House / Apartment	Occupation Select Occupation

Predict Approval

Conclusion:

The **Credit Card Approval Prediction System** successfully demonstrates how Machine Learning can assist financial institutions in evaluating credit card applications quickly and consistently. The application analyzes applicant information and predicts whether a credit card application is likely to be approved or rejected using a trained **XGBoost** Machine Learning model.

The project combines Python, Flask, Pandas, NumPy, Scikit-learn, XGBoost, HTML, CSS, and Joblib to develop a complete end-to-end web application. The trained model was integrated into a Flask web application and successfully deployed on **Render**, allowing users to access the system through a web browser.

This project enhanced practical knowledge in Machine Learning, data preprocessing, model development, Flask web application development, and cloud deployment. It also demonstrates how Machine Learning can improve decision-making processes in the banking and financial sector.

Future Scope

The Credit Card Approval Prediction System can be further enhanced in the following ways:

- Integrate real-time banking and credit score APIs for more accurate predictions.
- Improve prediction performance by training the model on larger and more diverse datasets.
- Implement user authentication and role-based access for secure application usage.
- Add explainable AI techniques to provide reasons behind prediction results.
- Enhance the application with dashboards, analytics, and prediction history for better decision support.
- Deploy the application using scalable cloud infrastructure for enterprise-level usage.